Learned Page Replacement under Dynamic Workload Distribution Shift: An Empirical Operating Systems Evaluation

Student Name: Md Shamim Ahmed Limon Student ID: 202414043
CSE-307: Operating Systems (Section B) Term Paper
Department of Computer Science and Engineering

October 2026

Abstract

Virtual memory management relies fundamentally on page replacement algorithms to minimize costly storage access latencies during page faults. While classical heuristic policies such as First-In, First-Out (FIFO) and Least Recently Used (LRU) rely on static assumptions regarding temporal and sequential locality, machine learning approaches aim to predict eviction candidates dynamically. In this study, we implement and benchmark four page replacement algorithms from scratch: FIFO, LRU, Belady's theoretical Optimal policy (MIN), and a lightweight Decision Tree-based Learned policy. Using a controlled synthetic benchmark of 2,000 page references with a deliberate midpoint distribution shift (transitioning from a locality-heavy working set to an erratic, bursty access pattern), we analyze algorithm resilience. Our experimental results show that in the locality-heavy phase, the Learned policy achieves a 59.60% hit ratio, outperforming LRU (53.20%) and FIFO (48.10%). However, when subjected to the random/bursty second phase, the static Learned policy's hit ratio drops to 13.30%, underperforming both LRU (21.60%) and FIFO (21.50%). These findings highlight the critical trade-off between predictive specialization and robustness to workload phase changes in learned operating system subsystems.

1 Introduction

Modern operating systems rely on paged virtual memory architectures to present applications with the abstraction of a vast, contiguous address space while operating within limited physical RAM [4]. When an active thread references a virtual page not currently resident in physical frames, the memory management unit triggers a hardware page fault exception, requiring the operating system to retrieve the missing page from backing store. Because secondary storage I/O is orders of magnitude slower than main memory access, page replacement algorithms strive to maximize the hit ratio and minimize fault frequencies [2].

Traditional replacement policies employ fixed heuristics. FIFO evicts the oldest resident page regardless of usage history, whereas LRU exploits temporal locality by discarding the page unaccessed for the longest duration [4]. Although Belady's Optimal algorithm [1] provides the theoretical upper bound on replacement efficiency by evicting the page needed furthest in the future, it requires omniscient future knowledge and is impossible to implement directly in real-time kernels.

Recently, learned operating systems components have garnered substantial attention [5]. By training machine learning models on offline execution traces, learned page replacement seeks to approximate Belady's decisions using historical state features. However, real-world application workloads undergo dramatic phase transitions, altering memory access distributions dynamically.

This paper investigates whether a lightweight, explainable Decision Tree policy can effectively learn page eviction patterns, how it compares against classical baselines, and how it responds when the underlying access distribution undergoes a sudden shift.

2 Background

2.1 Classical Replacement Policies

FIFO (First-In, First-Out): Allocates a circular or queue structure to record page arrival. Upon a fault in a full frame set, the page loaded earliest is evicted. FIFO ignores access frequency and recency, making it vulnerable to evicting heavily accessed pages (and susceptible to Belady's Anomaly) [4].

LRU (Least Recently Used): Exploits the principle of temporal locality by maintaining an ordering of access timestamps. When replacement is triggered, LRU evicts the page unreferenced for the longest time:

$$\text{victim}_{\text{LRU}} = \arg\min_{p \in \mathcal{F}} \big(\text{last_access}(p)\big) \tag{1}$$

where $\mathcal{F}$ represents the set of resident physical frames.

Optimal (Belady's MIN): Serves as an unachievable offline gold standard [1]. Belady proved that the minimum page fault rate is achieved by evicting the page whose next reference occurs furthest in the future:

$$\text{victim}_{\text{OPT}} = \arg\max_{p \in \mathcal{F}} \big(\text{next_access}(p)\big) \tag{2}$$

2.2 Learned Eviction Concept

A learned replacement policy models page eviction as a supervised classification problem [3, 5]. Whenever physi-

cal frames are exhausted and an eviction is mandatory, the model evaluates each resident page $p \in \mathcal{F}$ based strictly on past access statistics, predicting the probability that p aligns with the optimal victim.

3 Methodology

3.1 Experimental Architecture

To guarantee experimental integrity and avoid data leakage, our research methodology strictly isolates model training from evaluation, as illustrated in the pipeline architecture:

1. **Training Workloads:** 5 independent synthetic traces (seeds 100–104) are generated.
2. **Offline Oracle Generation:** Belady's algorithm simulates each training trace. At every eviction event, candidate feature vectors are extracted from past history and labeled ($y = 1$ for the optimal victim, $y = 0$ otherwise).
3. **Model Induction:** A `DecisionTreeClassifier` (maximum depth = 4, balanced class weights) is fitted to the offline dataset.
4. **Held-Out Evaluation:** A distinct evaluation workload (seed 42, length 2,000) containing an explicit midpoint shift is evaluated identically across FIFO, LRU, Optimal, and Learned policies.

3.2 Feature Formulation (Strict Past-Only)

During inference, the model must make real-time decisions without any lookahead into future steps. For each resident page $p \in \mathcal{F}$ at step t, we compute a 4-dimensional feature vector:

1. **Recency (Δt):** Steps elapsed since p was last accessed: $t - \text{last_access}(p)$.
2. **Total Frequency (f_{tot}):** Cumulative references to p since trace initialization.
3. **Sliding-Window Frequency (f_{win}):** References to p within the recent window $W = 50$ steps.
4. **Resident Age (a_{res}):** Steps elapsed since p entered physical memory: $t - \text{load_time}(p)$.

3.3 Inference and Deterministic Tie-Breaking

When a page fault occurs and $|\mathcal{F}| = K$ (allocated frames):

$$\text{victim} = \arg\max_{p \in \mathcal{F}} \Big(P(y = 1 \mid \mathbf{x}_p) \Big) \tag{3}$$

If multiple resident candidates share identical predicted probabilities, ties are broken deterministically by highest recency Δt, followed by lowest page identifier.

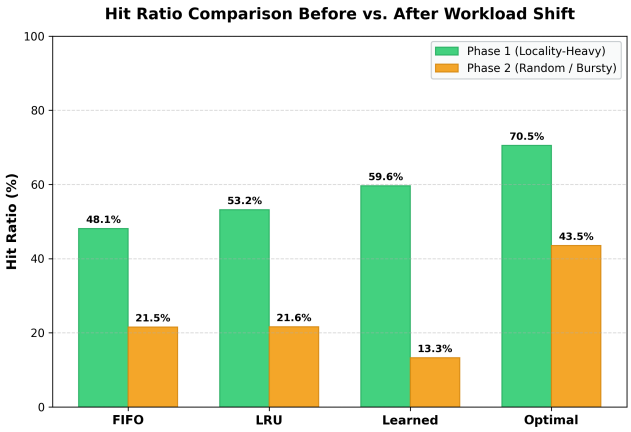


Figure 1: Hit ratio comparison across replacement algorithms before and after the midpoint workload shift ($K = 5$ frames).

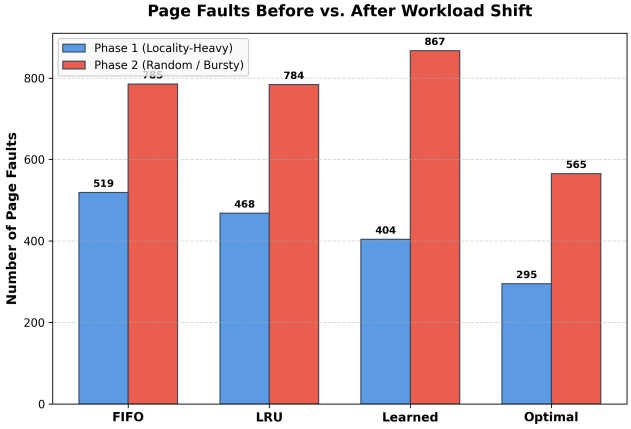


Figure 2: Total page fault counts before and after the workload shift.

3.4 Workload Shift Design

The evaluation workload comprises $N = 2000$ references with a total universe of $M = 50$ virtual pages, partitioned at shift index $S = 1000$:

- **Phase 1 (Steps 0–999, Locality-Heavy):** Accesses focus primarily on an active working set of 8 pages ($p \in [0, 7]$) with geometric repetition (65% probability), sequential loops (20%), and low noise (15%).
- **Phase 2 (Steps 1000–1999, Random / Bursty):** The working set dissolves into dynamic 6-page bursts that switch unpredictably across the full 50-page universe (45% probability), mixed with uniform random references across all 50 pages (55%).

4 Experimental Results

Table 1 displays the measured empirical performance across all algorithms for a standard memory allocation of $K = 5$ frames. All results satisfy the exact mathematical identity: hits + page_faults = references.

Table 1: Performance Metrics Segmented by Workload Phase

Algorithm	Phase	Refs	Hits	Faults	Hit Ratio
FIFO	Before Shift	1000	481	519	48.10%
	After Shift	1000	215	785	21.50%
	Overall	2000	696	1304	34.80%
LRU	Before Shift	1000	532	468	53.20%
	After Shift	1000	216	784	21.60%
	Overall	2000	748	1252	37.40%
Learned (Tree)	Before Shift	1000	596	404	**59.60%**
	After Shift	1000	133	867	13.30%
	Overall	2000	729	1271	36.45%
Optimal (MIN)	Before Shift	1000	705	295	**70.50%**
	After Shift	1000	435	565	43.50%
	Overall	2000	1140	860	**57.00%**

5 Discussion and Analysis

5.1 Phase 1: Learned Policy Performance

In the first half of the trace, memory accesses exhibit clear spatial and temporal structure. LRU achieves a 53.20% hit ratio by evicting pages with high recency values Δt. However, the Learned Decision Tree policy achieves a higher **59.60%** hit ratio on this trace, reducing page faults from 468 to 404.

The improvement is consistent with using several past-only signals rather than recency alone. However, aggregate hit/fault counts do not establish that a particular feature caused the gain, so the result should be interpreted as an empirical outcome of the complete learned policy rather than proof of a single causal mechanism.

5.2 Phase 2: Distribution Shift and Model Drift

Following step 1000, the access pattern shifts abruptly to unpredictable bursts across 50 pages. In this phase:

- Optimal hit ratio declines from 70.50% to 43.50%, consistent with the second phase having weaker locality and a much larger active page universe.
- LRU and FIFO also decline to 21.60% and 21.50% respectively.
- The static Learned policy drops further to **13.30%** hit ratio (867 faults).

The aggregate metrics alone do not prove that one feature caused this degradation. One plausible contributor is state-distribution mismatch: training labels are generated from resident-page states produced by the Optimal oracle, whereas during evaluation the learned policy's own earlier eviction decisions create different later candidate sets. The fixed tree is also not updated at the phase boundary. Because the training traces were generated from the same two-phase workload family, the second phase is best described as a workload phase shift rather than a completely unseen out-of-distribution workload.

5.3 Frame Capacity Sensitivity

We conducted an additional sensitivity analysis across frame counts $K \in \{3, 5, 7, 10\}$. At $K = 3$, the Learned policy reaches a 27.85% overall hit ratio versus LRU's 25.75%. At $K = 10$, however, LRU reaches 56.75% while the Learned policy falls to 33.60%. This shows that the learned policy is sensitive to frame capacity and that a model trained separately for each K does not necessarily scale monotonically with additional frames.

6 System and VM Resource Profiling Status

The repository includes `scripts/run_resource_profile.sh` and an environment logger for collecting elapsed time, CPU time, and maximum resident set size under Ubuntu/VMware. These scripts keep the algorithmic workload and random seed fixed while the student manually changes the VM resource configuration. At the time of this report draft, those VM runs have not been executed; therefore, no system-level performance conclusions are claimed here.

7 Conclusion

This study found that the learned policy outperformed FIFO and LRU during the structured first phase, but it did not outperform LRU overall and degraded sharply after the workload phase shift. Optimal remained the strongest offline reference because it uses future knowledge. The results motivate future work on online adaptation, periodic retraining, or hybrid fallbacks that combine learned decisions with robust classical policies.

References

[1] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.

[2] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[3] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[4] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley & Sons, 10th edition, 2018.

[5] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. Driving cache replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*, Boston, MA, July 2018. USENIX Association.